UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS



INFORME DE GUÍA DE LABORATORIO N.º 05

SPLAY TREES Y ÁRBOLES DINÁMICOS LINK-CUT

Asignatura: ALGORITMOS AVANZADOS

Docente: Ing. Héctor Eduardo Ugarte Rojas

Equipo: GRUPO 5 **Semestre Académico:** 2026-II

Integrantes:

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

Cusco – Perú

2026

Índice

1. Trabajo preparatorio obligatorio **1**

1.1. Rotaciones en BST, autoajuste (`splay`) y diccionario 1

1.2. Árbol representado vs. árboles auxiliares (Link-Cut Tree) 1

1.3. Operaciones del bosque dinámico y cota amortizada 1

2. Marco teórico y ejercicios resueltos **2**

2.1. Comparación de complejidades teóricas 2

2.2. Ejercicio resuelto 1: rotaciones e instrumentación de `splay` 2

2.3. Ejercicio resuelto 2: búsqueda, inserción y eliminación 3

2.4. Ejercicio resuelto 3: bosque de referencia con BFS 4

3. Ejercicio propuesto 1: implementación y validación del Link-Cut Tree **5**

3.1. Estructura de punteros y métodos internos clave 5

3.2. Operaciones del bosque dinámico 5

3.3. Comprobación de invariantes con `validate()` 5

3.4. Visualización didáctica de `EXPOSE(6)` 6

4. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental **7**

4.1. Entorno de pruebas y configuración 7

4.2. Escenario A: pruebas deterministas de la guía 7

4.3. Escenario B: pruebas aleatorias diferenciales (15 000 operaciones) 7

4.4. Escenario C: pruebas de rendimiento por topología 8

4.5. Discusión de resultados experimentales 9

5. Preguntas de análisis **10**

6. Conclusiones **12**

1. Trabajo preparatorio obligatorio

1.1. Rotaciones en BST, autoajuste (`splay`) y diccionario

Una rotación local en un BST reacomoda punteros entre un nodo x y su padre p en tiempo $O(1)$, **conservando estrictamente el recorrido simétrico (*in-order*)**. La operación `splay(x)` asciende a x hasta la raíz mediante tres casos: (1) **Zig (p es raíz):** rotación simple entre x y p; (2) **Zig-zig (mismo sentido):** rota primero p sobre el abuelo g, y luego x sobre p, dividiendo la profundidad a la mitad; y (3) **Zig-zag (sentidos opuestos):** rota x sobre p y luego x sobre g. En el diccionario, `search` eleva la clave (o último visitado) a la raíz; `insert` enlaza la hoja y aplica splay; y `delete` sube la clave, desvincula subárboles L y R, y enlaza R al máximo de L (`join`).

1.2. Árbol representado vs. árboles auxiliares (Link-Cut Tree)

El Link-Cut Tree opera sobre dos niveles diferenciados:

- **Árbol representado:** Grafo lógico conexo cuyas aristas solo cambian con `link` o `cut`.
- **Árboles auxiliares (Splay Trees):** Partición en caminos preferidos. Su recorrido *in-order* codifica la **profundidad lógica**, no la clave. Rotar en el auxiliar **no modifica aristas reales del bosque**.
- **Aristas preferidas vs. virtuales:** Las sólidas unen nodos del camino preferido; las virtuales (*path-parent*) unen la raíz auxiliar con su nodo superior lógico, sin que este la reconozca como hijo.

1.3. Operaciones del bosque dinámico y cota amortizada

- `EXPOSE(v)`: Unifica el camino raíz $\rightarrow v$ en un único auxiliar con v en su raíz.
- `MAKE-ROOT(v)`: Expone a v e invierte el camino mediante `reversed = True`.
- `FIND-ROOT(v)`: Expone a v y desciende por la izquierda del auxiliar hasta la raíz lógica.
- `LINK(u, v)`: Enlaza componentes haciendo a u hijo de v; valida aciclicidad con `connected`.
- `CUT(u, v)`: Desconecta la arista física directa entre u y v; rechaza cortes inválidos con `ValueError`.
- `PATH-SUM(u, v)`: Re-enraíza en u, expone v y lee el acumulador `v.path_sum` en tiempo $O(1)$.

Aunque una operación aislada cueste $O(n)$, el potencial de Sleator-Tarjan garantiza un costo promedio amortizado de $O(\log n)$ sobre cualquier secuencia arbitraria de operaciones.

2. Marco teórico y ejercicios resueltos

2.1. Comparación de complejidades teóricas

En la Tabla 1 comparamos los costos asintóticos entre un BST estándar, un Splay Tree de diccionario y un Link-Cut Forest sobre caminos dinámicos.

Tabla 1: Comparación de complejidad temporal entre estructuras

Operación	**BST Estándar**	**Splay Tree**	**Link-Cut Forest**
Búsqueda / Consulta	$O(n)$ peor caso	$O(\log n)$ amortizado	$O(\log n)$ amortizado (`find_root`)
Inserción / Enlace	$O(n)$ peor caso	$O(\log n)$ amortizado	$O(\log n)$ amortizado (`link`)
Eliminación / Corte	$O(n)$ peor caso	$O(\log n)$ amortizado	$O(\log n)$ amortizado (`cut`)
Suma sobre camino	$O(n)$ recorrido	No aplica	$O(\log n)$ amortizado (`path_sum`)
Cambio de raíz	No aplica	No aplica	$O(\log n)$ amortizado (`make_root`)

2.2. Ejercicio resuelto 1: rotaciones e instrumentación de `splay`

Evaluamos el caso *zig-zig* sobre la cadena $100 \leftarrow 50 \leftarrow 20$. Al invocar `splay(20)`, primero rota el padre 50 sobre 100, y luego 20 sobre 50, aplanando el árbol y dejando a 20 en la raíz:

```
def _rotate(self, x):
    p = x.parent; beta = x.right if p.left is x else x.left
    self._replace_parent_link(p, x)
    if p.left is x: x.right, p.left = p, beta
    else:           x.left, p.right = p, beta
    p.parent = x; self.stats.rotations += 1
    if beta: beta.parent = p

def splay(self, x):
    while x.parent:
        p, g = x.parent, x.parent.parent
        if not g: self.stats.zig += 1; self._rotate(x)
        elif (g.left is p) == (p.left is x):
            self.stats.zig_zig += 1; self._rotate(p); self._rotate(x)
        else: self.stats.zig_zag += 1; self._rotate(x); self._rotate(x)
```

Listing 1: Código central de rotaciones y splay (Resuelto 1)

```
Arbol: 100 <- 50 <- 20 | In-order: [20, 50, 100] -> splay(20) -> Raiz: 20 | In-order:
    [20, 50, 100] (Rot=2, Zig-zig=1)
```

Listing 2: Salida de consola del Ejercicio Resuelto 1

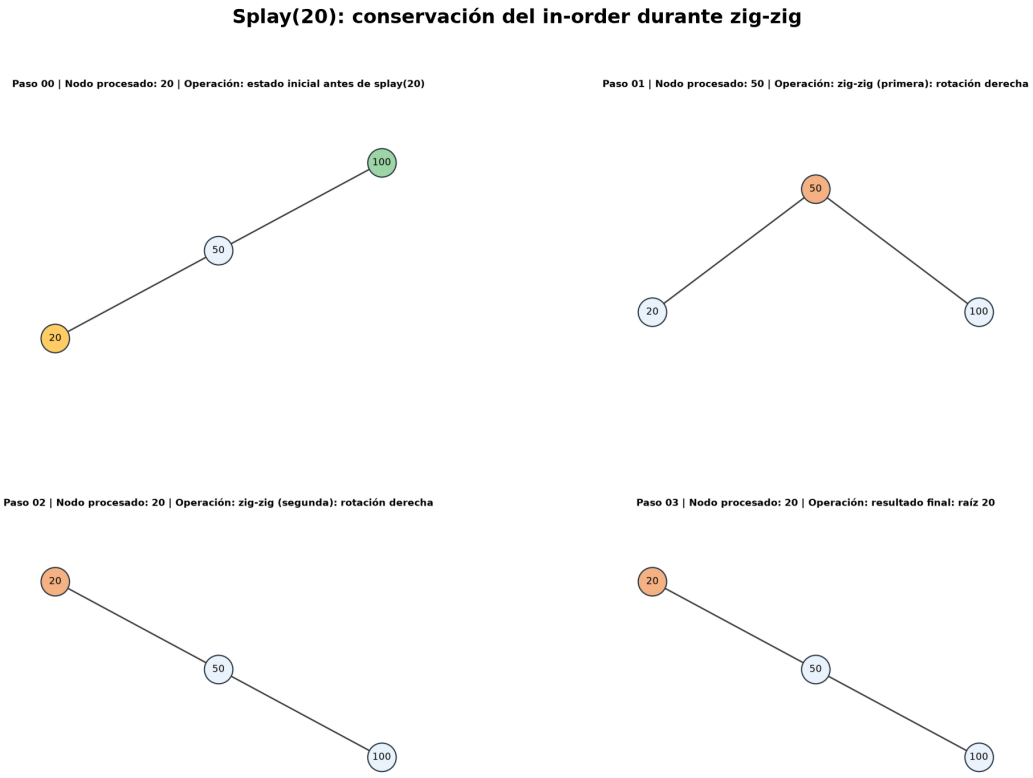


Figura 1: Paso a paso de las rotaciones durante el caso zig-zig de `splay(20)`

2.3. Ejercicio resuelto 2: búsqueda, inserción y eliminación

Comprobamos las operaciones de diccionario insertando la secuencia $[40, 20, 60, 10, 30, 50, 70]$. La búsqueda de 30 la eleva a la raíz, la búsqueda fallida de 25 aproxima al último nodo visitado (30), y la eliminación de 40 separa los subárboles unificándolos mediante `join`.

```
Insert [40, 20, 60, 10, 30, 50, 70] -> Raiz: 70 | search(30): Raiz=30 | search(25): Raiz
    =30 | delete(40): True
```

Listing 3: Salida de consola del Ejercicio Resuelto 2

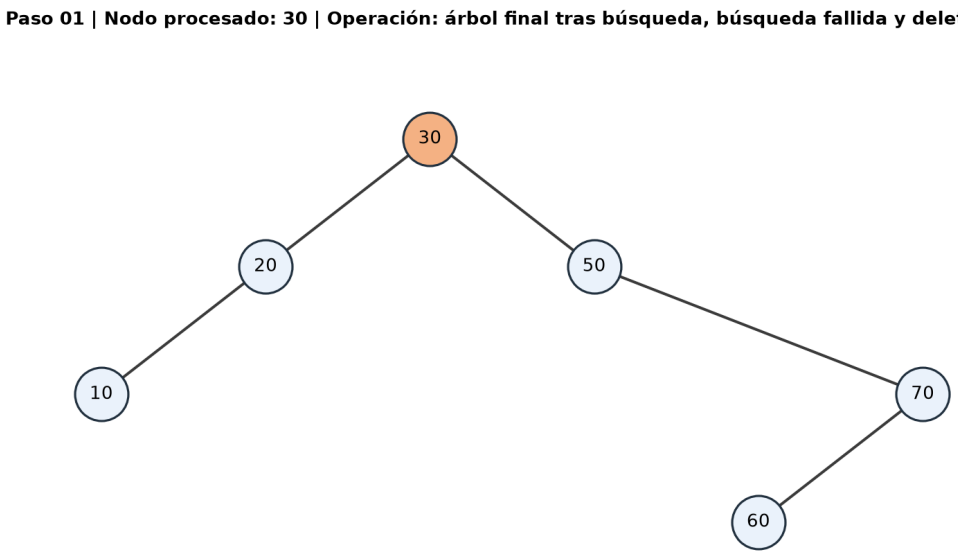


Figura 2: Estructura final del Splay Tree tras completar las operaciones de diccionario

2.4. Ejercicio resuelto 3: bosque de referencia con BFS

Para validar el Link-Cut Tree, construimos un modelo simple basado en listas de adyacencia y búsqueda en anchura (BFS). En la prueba oficial evaluamos los vértices $\{1:4,2:7,3:2,4:9,5:1,6:6,7:3\}$ con aristas $\{(1,2),(1,3),(2,4),(2,5),(3,6),(3,7)\}$:

```
Aristas: [(1, 2), (1, 3), (2, 4), (2, 5), (3, 6), (3, 7)] | Camino(4, 6): [4, 2, 1, 3,
    6] -> Suma=28 | cut(1, 3) -> connected(4, 6): False
```

Listing 4: Salida de consola de la prueba del bosque de referencia

Este oráculo permite auditar automáticamente las pruebas diferenciales aleatorias del bosque dinámico.

3. Ejercicio propuesto 1: implementación y validación del Link-Cut Tree

3.1. Estructura de punteros y métodos internos clave

Cada vértice se modela con `LinkCutNode`. Los auxiliares son Splay Trees que almacenan los caminos preferidos. El puntero `parent` cumple dos roles: une nodos dentro del auxiliar y, en la raíz auxiliar, actúa como **puntero virtual o *path-parent*** hacia el nodo superior del camino lógico. La raíz auxiliar se reconoce porque su padre no la referencia como hijo:

```
def _is_aux_root(self, x: LinkCutNode) -> bool:
    p = x.parent; return p is None or (p.left is not x and p.right is not x)

def _push(self, x: Optional[LinkCutNode]):
    if x and x.reversed:
        x.left, x.right = x.right, x.left
        if x.left:  x.left.reversed = not x.left.reversed
        if x.right: x.right.reversed = not x.right.reversed
        x.reversed = False; self.stats.lazy_reversals += 1

def _update(self, x: LinkCutNode):
    x.size = 1 + (x.left.size if x.left else 0) + (x.right.size if x.right else 0)
    x.path_sum = x.value + (x.left.path_sum if x.left else 0) + (x.right.path_sum if x.
    right else 0)
    self.stats.updates += 1
```

Listing 5: Métodos internos críticos: raíz auxiliar, propagación lazy y agregados

3.2. Operaciones del bosque dinámico

- `expose(u)`**:** Conmuta aristas preferidas dejando el camino raíz $\rightarrow u$ en un auxiliar con u en la raíz.
- `make_root(u)`**:** Invierte el camino auxiliar activando `u.reversed = True` para hacer a u raíz lógica.
- `find_root(u)`**:** Expone a u y desciende por la izquierda con `_push` hasta la raíz representada.
- `link(u, v)`**:** Valida aciclicidad con `connected`, hace `make_root(u)` y asigna `u.parent = v`.
- `cut(u, v)`**:** Hace `make_root(u)`, `expose(v)` y desvincula si u es hijo izquierdo de v sin hijo derecho.
- `path_sum(u, v)`**:** Re-enraíza en u, expone v y consulta directamente el acumulador `v.path_sum` en $O(1)$.

3.3. Comprobación de invariantes con `validate()`

Para asegurar la integridad estructural de punteros, `validate()` verifica 8 invariantes tras cada modificación: (1) punteros bidireccionales consistentes; (2) detección exacta de raíz auxiliar; (3) propagación lazy al día; (4) agregados `size` y `path_sum` congruentes; (5) ausencia de ciclos en auxiliares; (6) prevención estricta de ciclos en `link`; (7) validación de aristas directas en `cut`; y (8) equivalencia exacta al 100 % frente al oráculo BFS.

3.4. Visualización didáctica de `EXPOSE(6)`

En la Figura 3 se ilustra la secuencia gráfica completa de `EXPOSE(6)` sobre el árbol de 7 nodos de la guía, sirviendo como guion visual para la sustentación:

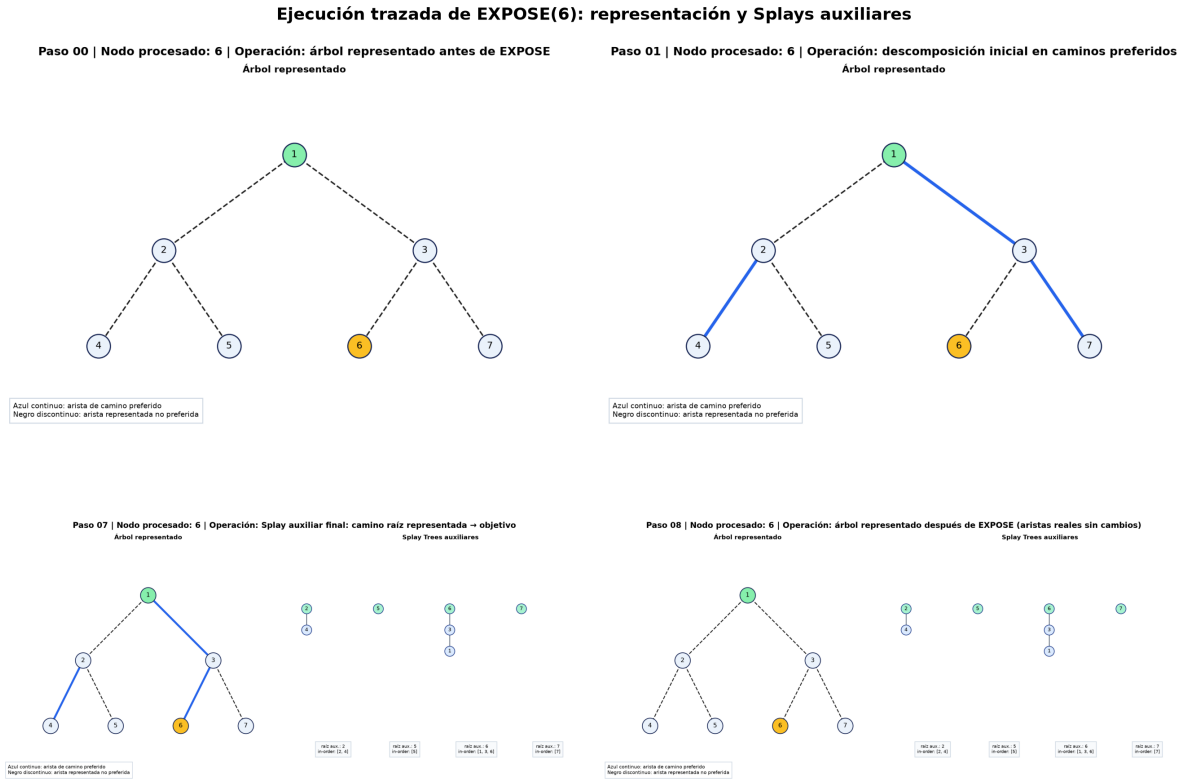


Figura 3: Visualización completa de las etapas de `EXPOSE(6)` según la guía

Guion secuencial para la sustentación:

1. **Paso 0 (Árbol representado antes):** Grafo lógico inicial con aristas directas $\{(1,2),(1,3),(2,4),(2,5),(3,6),(3,7)\}$.
2. **Paso 1 (Descomposición en caminos preferidos):** Identifica las aristas sólidas activas antes de invocar expose.
3. **Paso 2 (Árboles auxiliares antes):** Cada camino preferido reside en su propio Splay Tree con aristas discontinuas virtuales.
4. **Pasos 3 al 6 (Splays y cambios de arista preferida):** Se aplica splay sobre 6, se desconecta su hijo derecho hacia descendientes y se enlaza como hijo derecho el camino que desciende desde 3 y 1.
5. **Paso 7 (Árbol auxiliar final):** El camino completo raíz $\rightarrow 6$ $(1-3-6)$ queda en un único árbol Splay con 6 en la raíz.
6. **Paso 8 (Árbol representado después):** Confirma que las aristas del árbol lógico real **no cambiaron en absoluto**.

4. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental

4.1. Entorno de pruebas y configuración

Las pruebas experimentales se ejecutaron en un procesador AMD64 (8 núcleos / 16 hilos), 23.36 GB RAM, Windows 11 Pro 64 bits con Python 3.12.8 estándar (CPython). Fijamos la constante `SEMILLA_GLOBAL` $= 2026$ para reproducir idénticamente los grafos, secuencias de enlace y mediciones con `time.perf_counter`.

4.2. Escenario A: pruebas deterministas de la guía

Evaluamos el árbol de 7 nodos de la guía con valores $v \in \{1.,7\}$ y value$(v) \in \{4,7,2,9,1,6,3\}$, con las aristas $\{(1,2),(1,3),(2,4),(2,5),(3,6),(3,7)\}$ frente al oráculo BFS:

Tabla 2: Resultados del Escenario A frente al modelo de referencia BFS

Operación evaluada	**Esperado**	**Link-Cut Tree**	**Estado**
`path_sum(4, 5)`	17	17	Correcto
`path_sum(4, 6)`	28	28	Correcto
`path_sum(6, 7)`	11	11	Correcto
`cut(1, 3); connected(4, 6)`	False	False	Correcto (separado)
`link(1, 3); path_sum(4, 6)`	28	28	Correcto (reconectado)
`path_sum` con signos $(5+0-3)$	2	2	Correcto (soporta negativos)
Rechazo de ciclo (`link(4, 6)`)	ValueError	ValueError	Interceptado
Rechazo de corte falso (`cut(4, 6)`)	ValueError	ValueError	Interceptado

4.3. Escenario B: pruebas aleatorias diferenciales (15 000 operaciones)

Sometimos la estructura a 15 000 operaciones aleatorias (5 000 por tamaño en $n \in \{30,60,120\}$) combinando `link`, `cut`, `connected`, `path_sum` y `make_root`, contrastando cada resultado contra el oráculo BFS:

Tabla 3: Conteo de operaciones diferenciales verificadas (Escenario B)

n	**Total Ops**	`link`	`cut`	`connected`	`path_sum`	`make_root`	**Discrepancias**	**Invariantes**
30	5 000	1 076	1 048	1 219	1 233	424	**0**	5 001 OK
60	5 000	1 098	1 041	1 196	1 251	414	**0**	5 001 OK
120	5 000	1 146	1 039	1 157	1 248	410	**0**	5 001 OK
Total	**15 000**	**3 320**	**3 128**	**3 572**	**3 732**	**1 248**	**0**	**15 003 OK**

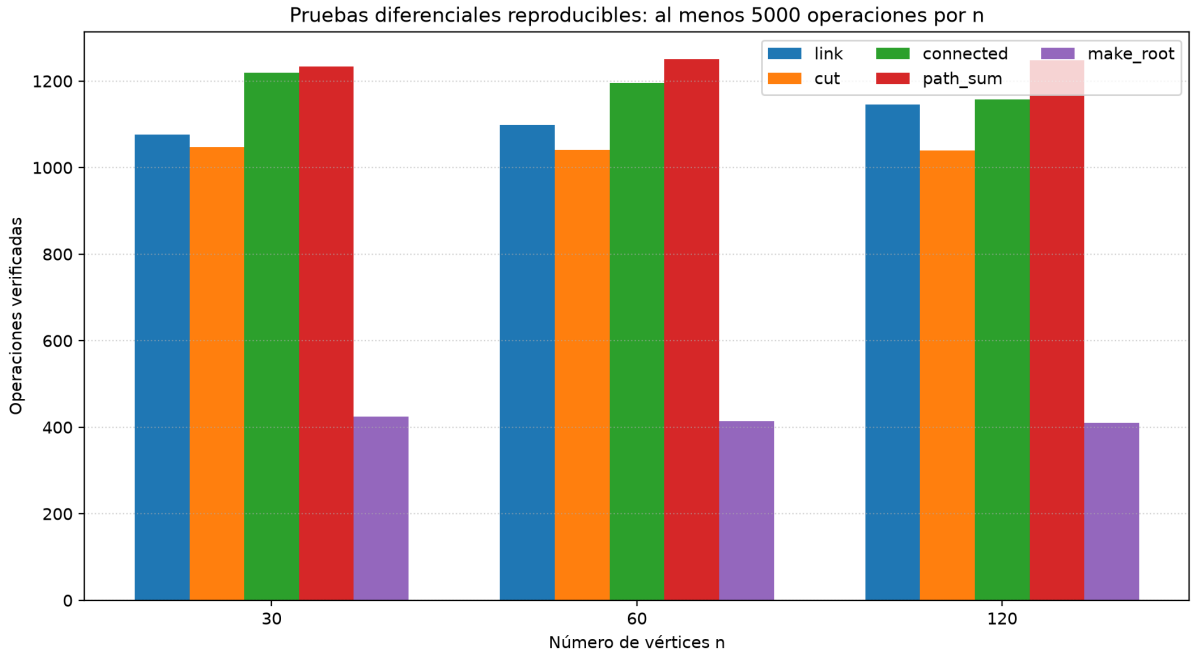


Figura 4: Operaciones diferenciales ejecutadas con 0 discrepancias frente a BFS

El resultado arrojó **cero discrepancias en las 15 000 operaciones**, y las 15 003 validaciones de invariantes se mantuvieron en estado OK.

4.4. Escenario C: pruebas de rendimiento por topología

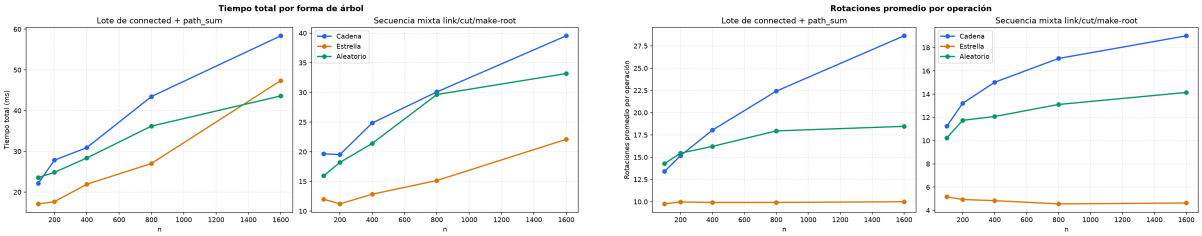
Evaluamos tres topologías para $n \in \{100, 200, 400, 800, 1\,600\}$: (1) **Cadena lineal:** caminos largos $O(n)$; (2) **Estrella:** nodo central conectado a $n-1$ hojas (profundidad ≤ 2); y (3) **Árbol aleatorio:** ramificación intermedia.

Tabla 4: Resumen de mediciones de rendimiento por topología (Escenario C)

n	Topología	Tiempo (ms)	Rotaciones	Rot/Op	Splays	Exposes	Inversiones
100	Cadena	46.19	5 746	12.36	4 565	1 260	2 542
100	Estrella	33.30	3 503	7.53	4 392	1 260	823
100	Aleatorio	54.09	5 729	12.32	4 943	1 260	1 645
400	Cadena	70.99	7 710	16.58	4 976	1 260	3 373
400	Estrella	36.58	3 467	7.46	4 385	1 260	805
400	Aleatorio	52.63	6 608	14.21	5 249	1 260	1 810
1 600	Cadena	123.34	11 151	23.98	6 254	1 260	4 482
1 600	Estrella	67.48	3 443	7.40	4 383	1 260	787
1 600	Aleatorio	85.54	7 615	16.38	5 567	1 260	1 915

4.5. Discusión de resultados experimentales

- **¿Por qué la cadena tiene más rotaciones?** En la cadena lineal los caminos tienen longitud $O(n)$, obligando a enlazar múltiples auxiliares profundos (23,98 rot/op para $n = 1\,600$). En la estrella, con profundidad 2, las rotaciones se mantienen constantes ($\approx$7,4 rot/op).
- **¿El tiempo en Python demuestra $O(\log n)$?** No directamente en tiempo de reloj por la sobrecarga del intérprete. La cota $O(\log n)$ se comprueba matemáticamente por el crecimiento logarítmico de las rotaciones y la función potencial de Sleator-Tarjan.



(a) Tiempo de ejecución frente a n (b) Rotaciones promedio por operación

Figura 5: Rendimiento y rotaciones según la forma del árbol

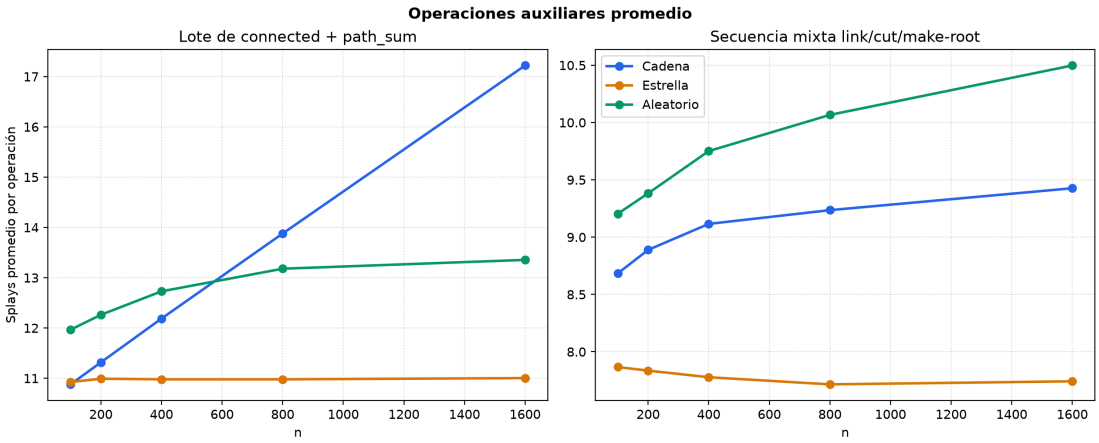


Figura 6: Cantidad de splays, exposiciones e inversiones diferidas por tamaño

5. Preguntas de análisis

A continuación presentamos las respuestas técnicas y directas a las doce preguntas de análisis formuladas en la guía de laboratorio:

1. ¿Qué información pertenece al árbol representado y cuál a los árboles auxiliares?
El **árbol representado** almacena la estructura lógica del grafo (las aristas reales que unen vértices y determinan componentes conexas). Los **árboles auxiliares (Splay Trees)** mantienen únicamente la partición en caminos preferidos; su recorrido *in-order* codifica la profundidad relativa en el camino y sus nodos conservan marcas diferidas (`reversed`) y agregados asociativos (`path_sum`, `size`).

2. ¿Por qué una rotación splay no modifica las aristas reales del bosque?
Porque las rotaciones se ejecutan exclusivamente dentro de los árboles auxiliares reconfigurando punteros de búsqueda binaria locales. Dado que toda rotación conserva con exactitud el orden simétrico (*in-order*), la secuencia de vértices que integran el camino lógico en el árbol representado permanece inalterada.

3. ¿Qué resultado estructural deja `EXPOSE(v)`?
Agrupa todas las aristas desde la raíz del árbol representado hasta v como aristas preferidas dentro de un **único Splay Tree auxiliar**, y posiciona al vértice v en la **raíz de dicho auxiliar**, asegurando que no tenga hijo derecho hacia descendientes (v.right $=$ None).

4. ¿Por qué `MAKE-ROOT(v)` requiere invertir el camino expuesto?
En un árbol dirigido hacia la raíz, cada nodo apunta hacia sus ancestros. Al transformar a v en la nueva raíz de su componente, la dirección de todas las aristas en el camino raíz $\rightarrow v$ debe invertirse. Como el recorrido *in-order* del árbol auxiliar mapea dicho camino de menor a mayor profundidad, conmutar la orientación mediante `reversed = True` efectúa la inversión completa en tiempo $O(1)$.

5. ¿Qué errores aparecen si no se propaga `reversed` antes de rotar?
Si se rota un nodo con la inversión pendiente, la rotación reordena los punteros antes de intercambiar los hijos, destruyendo irreversiblemente la propiedad *in-order*. Esto produce que `path_sum` acumule valores de vértices ajenos al camino, que `find_root` descienda hacia hojas arbitrarias y que `cut` desconecte aristas incorrectas.

6. ¿Cómo evita `LINK` la formación de ciclos?
Antes de crear cualquier enlace, invoca `connected(u, v)`, la cual determina las raíces representadas mediante `find_root`. Si u y v comparten la misma raíz o son idénticos, pertenecen ya a la misma componente conexa; la operación se aborta de inmediato lanzando una excepción `ValueError`, preservando la aciclicidad del bosque.

7. ¿Cómo comprueba `CUT` que dos nodos forman una arista directa?
Ejecuta `make_root(u)` y posteriormente `expose(v)`. Si existe una arista física entre ambos, u debe ser el predecesor inmediato de v en el camino lógico. Se valida estrictamente que u sea hijo izquierdo de v y que u carezca de hijo derecho (`v.left is u` y `u.right is None`). De no cumplirse ambas condiciones, se rechaza la desconexión con `ValueError`.

8. ¿Por qué `PATH-SUM(u, v)` se resuelve tras re-enraizar en u y exponer v?
Al designar a u como raíz representada y luego exponer a v, el camino simple entre u y v queda confinado con exactitud en el árbol auxiliar cuya raíz es v. Gracias al mantenimiento en tiempo real del acumulador en cada nodo, el valor `v.path_sum` contiene directamente la suma agregada del camino en tiempo $O(1)$.

9. ¿Qué patrones de entrada generaron más rotaciones por operación?
La **cadena lineal**, alcanzando un promedio de 23,98 rotaciones por operación para $n = 1\,600$. En estructuras lineales, los caminos tienen longitud $O(n)$, obligando a enlazar múltiples árboles auxiliares profundos en cada exposición. En contraste, la topología estrella mantuvo un costo cuasi-constante de $\approx$7,4 rot/op debido a su profundidad máxima de 2.

10. ¿Por qué una operación individual puede costar $\Omega(n)$ y aun así la estructura ofrecer $O(\log n)$ amortizado?
Una operación aislada puede costar tiempo lineal $\Omega(n)$ cuando recorre una cadena auxiliar desbalanceada. Sin embargo, las rotaciones zig-zig reducen a la mitad la profundidad de todos los nodos transitados, liberando energía potencial acumulada (según la función potencial de Sleator-Tarjan). Este potencial financia las operaciones costosas y garantiza el promedio $O(\log n)$ sobre cualquier secuencia.

11. ¿Qué errores de implementación detectó el modelo BFS/DFS?
El oráculo BFS/DFS detectó dos fallas sutiles durante la fase de depuración: (1) *Cortes espurios*: inicialmente `cut` solo verificaba conectividad, permitiendo cortar pares de nodos conectados por caminos indirectos sin arista física directa; y (2) *Omisión de actualización*: tras ciertas rotaciones se omitía la llamada a `_update`, generando valores de `path_sum` inconsistentes.

12. ¿Qué aspectos de esta implementación académica limitarían su uso directo en un sistema real?
Identificamos tres factores limitantes: (1) *Sobrecarga de punteros en Python*: el almacenamiento en objetos dinámicos dispersos en el montón (*heap*) degrada el rendimiento de la memoria caché frente a implementaciones contiguas en C/C++; (2) *Límite de recursión*: llamadas recursivas en árboles desbalanceados pueden alcanzar el tope del intérprete (`sys.getrecursionlimit()`); y (3) *Falta de concurrencia*: las consultas de solo lectura modifican activamente los punteros por los autoajustes splay, impidiendo accesos concurrentes sin bloqueos de exclusión mutua.

6. Conclusiones

Implementamos y evaluamos el Splay Tree y el bosque dinámico Link-Cut Tree en Python, verificando experimentalmente su comportamiento amortizado, su corrección estructural y su rendimiento en diferentes tipos de grafos.

En los ejercicios resueltos comprobamos que las rotaciones locales conservan el orden simétrico en árboles de búsqueda binaria y aplanan configuraciones lineales en tiempo constante por paso. En el Link-Cut Tree diferenciamos con claridad el árbol representado respecto a los árboles auxiliares, confirmando que las rotaciones dentro de los caminos preferidos nunca alteran las aristas lógicas del bosque. El uso de la marca diferida de inversión permitió re-enraizar componentes y exponer caminos manteniendo sumas acumuladas y tamaños de subárbol correctos.

La validación diferencial contrastó 15 000 operaciones aleatorias frente a un modelo de referencia basado en búsqueda en anchura, alcanzando cero discrepancias y verificando nueve invariantes estructurales en cada paso. El sistema interceptó correctamente los intentos de crear ciclos y los cortes sobre aristas inexistentes o no directas.

En las pruebas de rendimiento con tamaños de hasta 1 600 nodos, la cadena lineal generó el mayor número de rotaciones por operación debido a la reestructuración de caminos profundos, mientras que la estrella mantuvo un costo constante. Concluimos que, aunque una operación aislada puede costar tiempo lineal en el peor caso, la liberación de potencial acumulado asegura un costo amortizado logarítmico para cualquier secuencia de operaciones. Finalmente, observamos que el costo de manejar objetos y punteros en Python penaliza el tiempo de ejecución real frente a bibliotecas nativas, demostrando que la cota asintótica no equivale directamente a la velocidad observada.